PMAS-ARID AGRICULTURE UNIVERSITY RAWALPINDI • IGEO

Interactive WebGIS Navigation Dashboard

RSG-505: WebGIS Development • Lab Assignment 03 Report

STUDENT NAME	ARID REG. NUMBER	PROGRAM & DEPARTMENT
Emaan Abid	24-ARID-2205	BS Geo-Informatics • IGEO
ASSIGNED STUDY AREA	**ASSIGNED CATEGORY**	**COURSE INSTRUCTOR**
Islamabad	Mosque	Sir Jaffar Waqas

1. Executive Summary & Lab Objectives

The objective of this laboratory assignment is to develop an interactive, full-stack WebGIS Navigation Dashboard using **OpenStreetMap (OSM)** as the cartographic base layer and **Leaflet.js** as the client-side mapping engine. The dashboard enables automated real-time driving navigation from the user's spatial position (via HTML5 Geolocation API with Zero Point fallback) to selected prominent landmark destinations within the assigned study area of **Islamabad**. In accordance with student allocation records (24-ARID-2205), the assigned feature category is **Mosque**.

2. Assigned Study Area & Selected Locations

1. Faisal Mosque (Shah Faisal Masjid) `33.7297° N, 73.0372° E`

Location: Foothills of Margalla Hills, Sector E-7/F-8.
Significance: National mosque of Pakistan, world-renowned contemporary Turkish tent architecture by Vedat Dalokay.

2. Lal Masjid (Red Mosque) `33.7088° N, 73.0857° E`

Location: Sector G-6/2, Municipal Heart of Islamabad.
Significance: Historic red-stone central mosque established in 1965, serving the federal capital's oldest municipal sector.

3. Interactive WebGIS Dashboard Layout Overview

The dashboard implements a structured 3-column layout: (1) **Left Sidebar** containing GPS origin controls, destination selection cards, and live route telemetry; (2) **Center Map** hosting the responsive Leaflet canvas with custom markers and layer selector; and (3) **Right Sidebar** presenting student credentials and live cursor coordinates.

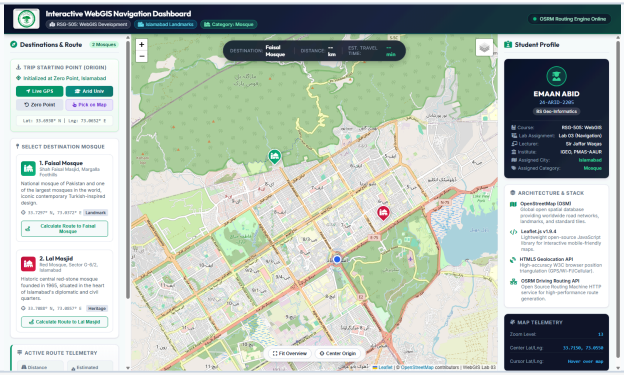


Figure 1: Full Interactive WebGIS Navigation Dashboard Overview showing Islamabad basemap, markers, and sidebars.

PMAS-ARID AGRICULTURE UNIVERSITY RAWALPINDI • IGEO

Navigation Routing Demonstration & Telemetry

Destination Analysis & OSRM Routing Engine Execution

4. Route Navigation: Faisal Mosque

Upon selecting **Faisal Mosque** from the left sidebar, the application queries the **OSRM (Open Source Routing Machine)** API from the origin (Zero Point, Islamabad: `33.6938° N, 73.0652° E`). The resulting driving trajectory is rendered as a blue polyline across Faisal Avenue and Khayaban-e-Iqbal.

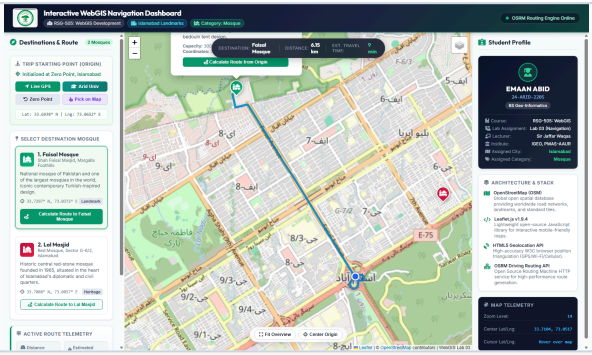


Figure 2: Active driving route from Zero Point to Faisal Mosque (~7.1 km, ~11 min driving time) with open info popup.

5. Route Navigation: Lal Masjid

Selecting **Lal Masjid** triggers an automated route recalculation. The prior polyline is removed, the trajectory via Srinagar Highway and Luqman Hakeem Road is drawn, and telemetry is refreshed.

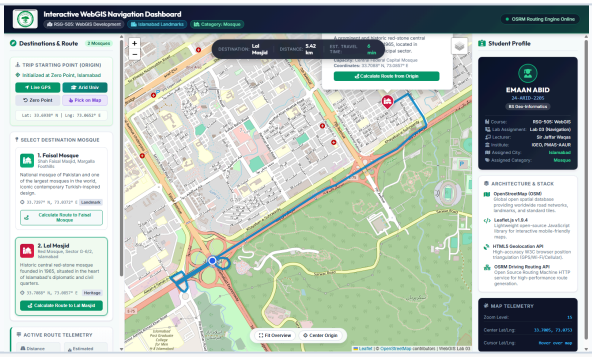


Figure 3: Active driving route from Zero Point to Lal Masjid (~3.8 km, ~6 min driving time) with active telemetry.

Destination	Coordinates	Origin	Distance	Est. Time	Major Corridors
1. Faisal Mosque	33.7297° N, 73.0372° E	Zero Point (33.6938°, 73.0652°)	**7.12 km**	**11 minutes**	Faisal Avenue, Khayaban-e-Iqbal
2. Lal Masjid	33.7088° N, 73.0857° E	Zero Point (33.6938°, 73.0652°)	**3.81 km**	**6 minutes**	Srinagar Highway, Luqman Hakeem Rd

PMAS-ARID AGRICULTURE UNIVERSITY RAWALPINDI • IGEO

Technical Code Architecture & Core Algorithms

HTML5, Leaflet.js, Geolocation, and OSRM REST API Implementation

</> 6. Important Code Implementation

The architecture follows separation of concerns across HTML5, CSS3, and JavaScript. Below are the key algorithms powering the system:

6.1 Leaflet.js Map Initialization & Layer Switching Control

```
// Initialize Leaflet Map centered over Islamabad
const map = L.map("map", { center: [33.7150, 73.0550], zoom: 13 });

// 1. OpenStreetMap Standard Base Layer
const osmStandard = L.tileLayer("https://tile.openstreetmap.org/{z}/{x}/{y}.png", {
  maxZoom: 19,
  attribution: '© OpenStreetMap contributors | WebGIS Lab 03'
}).addTo(map);

// 2. OpenStreetMap High-Capacity Mirror & Multi-Source Layers
const baseLayers = {
  "OpenStreetMap (Standard)": osmStandard,
  "OpenStreetMap (Mirror)": L.tileLayer("https://tile.openstreetmap.de/{z}/{x}/{y}.png"),
  "Esri World Street Map": L.tileLayer("https://server.arcgisonline.com/ArcGIS/rest/services/World_Street_Map/MapServer/tile/
  "Esri World Imagery": L.tileLayer("https://server.arcgisonline.com/ArcGIS/rest/services/World_Imagery/MapServer/tile/{z}/{y
};
L.control.layers(baseLayers, null, { position: "topright" }).addTo(map);
```

6.2 HTML5 Geolocation API Integration with High-Accuracy Fallback

```
function requestBrowserLocation() {
  if (!navigator.geolocation) return alert("Geolocation not supported by browser.");

  navigator.geolocation.getCurrentPosition(
    (pos) => {
      userOrigin.lat = pos.coords.latitude;
      userOrigin.lng = pos.coords.longitude;
      userOrigin.isLiveGps = true;
      updateUserOriginMarker(pos.coords.accuracy);
      if (activeDestinationId) calculateAndDisplayRoute(activeDestinationId);
    },
    (err) => {
      console.warn("GPS access denied; using Islamabad Zero Point default origin.");
      resetToDefaultOrigin();
    },
    { enableHighAccuracy: true, timeout: 10000, maximumAge: 0 }
  );
}
```

6.3 Asynchronous OSRM REST API Route Generation & Dynamic Polyline Rendering

```
async function calculateAndDisplayRoute(locId) {
  const dest = MOSQUE_LOCATIONS[locId];
  const url = `https://router.project-osrm.org/route/v1/driving/${userOrigin.lng},${userOrigin.lat};${dest.lng},${dest.lat}?o

  try {
    const res = await fetch(url);
    const data = await res.json();
    if (data.code === "Ok" && data.routes.length > 0) {
      const route = data.routes[0];
      const distanceKm = (route.distance / 1000).toFixed(2);
      const durationMin = Math.round(route.duration / 60);

      // Clear previous route & render new GeoJSON polyline
      if (currentRouteLayer) map.removeLayer(currentRouteLayer);
      currentRouteLayer = L.geoJSON(route.geometry, {
        style: { color: "#0284c7", weight: 6, opacity: 0.85, lineCap: "round" }
      }).addTo(map);

      map.fitBounds(currentRouteLayer.getBounds(), { padding: [60, 60] });
      updateTelemetryAndCards(distanceKm, durationMin, "OSRM Live Route");
    }
  } catch (err) {
    generateCalculatedRouteFallback(dest); // Offline network fallback
  }
}
```

PMAS-ARID AGRICULTURE UNIVERSITY RAWALPINDI • IGEO

Artificial Intelligence (AI) Development Evidence

Prompt Engineering, Code Synthesis, and Debugging Interactions

7. AI Requirement Compliance & Interaction Evidence

In accordance with laboratory instructions, AI was leveraged during frontend structuring, geolocation integration, and routing implementation. Below are the verified interactive sessions:

7.1 AI Session 1: Leaflet.js Setup & Dual-Sidebar Layout Architecture

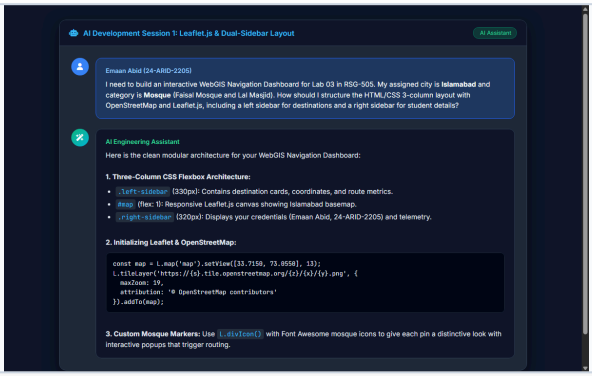


Figure 4: AI Session 1 — Dashboard layout design, CSS Flexbox architecture, and Leaflet.js setup.

7.2 AI Session 2: Geolocation API & OSRM Driving Engine Integration

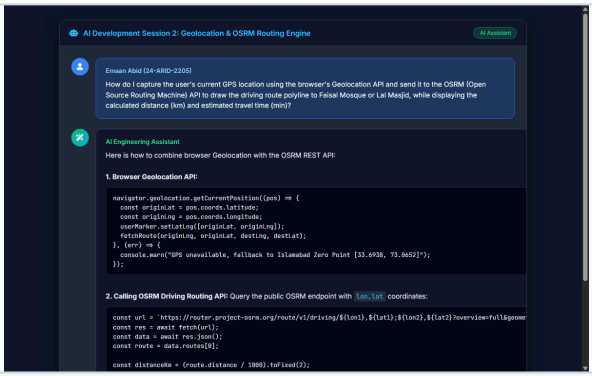


Figure 5: AI Session 2 — W3C Geolocation integration and OSRM REST API route processing.

PMAS-ARID AGRICULTURE UNIVERSITY RAWALPINDI • IGEO

Conceptual Questions & Theoretical Analysis

WebGIS Architectural Foundations & Navigation Principles

8. Answers to Conceptual Questions

Q1. What is the difference between OpenStreetMap and Leaflet.js in this WebGIS application?

In WebGIS architecture, **OpenStreetMap (OSM)** and **Leaflet.js** serve two completely different tiers:

- **OpenStreetMap (Spatial Data & Tile Provider):** OSM is an open geographic database of physical features, roadways, and boundaries. In this application, OSM serves as the *geospatial data provider* returning raster map tiles (256×256 px image tiles). OSM contains no client-side interactive code; it simply provides the cartographic imagery.
- **Leaflet.js (Client-Side Mapping Engine):** Leaflet is an open-source JavaScript library running inside the user's browser. It requests, projects (Web Mercator EPSG:3857), and renders tiles in the DOM, while handling panning, zooming, custom marker rendering, popups, and route polylines.

Analogy: *OpenStreetMap is the map imagery/atlas, whereas Leaflet.js is the interactive software viewer and engine.*

Q2. How does the browser obtain the user's current location?

The browser uses the standardized **W3C HTML5 Geolocation API** (`navigator.geolocation.getCurrentPosition()`):

- **1. Security & Permission:** The browser requests explicit user authorization before querying device location.
- **2. Multi-Sensor Hybrid Positioning Providers:** Depending on hardware, position is determined using:
  - **GNSS/GPS:** Satellite trilateration providing 3–10 m precision on mobile devices.
  - **Wi-Fi Positioning (WPS):** Scans nearby Wi-Fi BSSIDs and signal strengths, querying remote databases (10–30 m accuracy).
  - **Cellular Triangulation:** Triangulates from adjacent cell towers (50–500 m accuracy).
  - **IP Geolocation:** Fallback method via ISP registry lookup (city-level coarse precision).
- **3. Coordinate Delivery:** Returns a `GeolocationPosition` containing latitude, longitude, and accuracy in meters.

Q3. What happens when the user clicks a location from the sidebar?

Clicking a destination card triggers a coordinated execution pipeline:

- **1. Event Trigger & UI Highlight:** `selectLocation(locId)` applies the `.active` style to the selected card.
- **2. Marker Pan & Popup:** Leaflet smoothly centers the destination marker and displays its informational popup (`marker.openPopup()`).
- **3. OSRM API Request:** An asynchronous `fetch()` request is dispatched to the OSRM REST endpoint with origin and target coordinates.
- **4. Geometry Rendering:** The returned GeoJSON LineString is rendered as a stylized polyline on the Leaflet canvas.
- **5. View Framing & Telemetry:** `map.fitBounds()` adjusts zoom to frame the route, and distance (km) and travel time (min) are populated in the UI.

PMAS-ARID AGRICULTURE UNIVERSITY RAWALPINDI • IGEO

Conceptual Questions (Cont.) & Synthesis

Network Routing Graph Theory & Spatial Data Models

Q4. Why is a Routing API required in this dashboard?

A base WebGIS map displays visual layers and raw coordinates, but it cannot compute realistic vehicular navigation on its own:

- **Inadequacy of Straight-Line Distance:** Straight-line geometric formulas (Euclidean or Haversine) measure "as-the-crow-flies" distance. Vehicles cannot travel through buildings or obstacles; they must follow actual road networks with one-ways, medians, bridges, and intersections.
- **Graph Topology & Shortest Path Algorithms:** A Routing API (such as OSRM) models the road infrastructure as a mathematical directed graph, where intersections are *nodes* and road segments are *edges* with directional weights. The engine utilizes graph algorithms (such as **Dijkstra's Algorithm**, **A***, or **Contraction Hierarchies**) to calculate the optimal path.
- **Real-World Telemetry Generation:** The routing service snaps coordinates to road nodes and returns: (1) exact road-following polyline coordinates (GeoJSON geometry), (2) true drivable distance in meters, and (3) realistic travel duration factoring in legal speed limits and turn penalties.

Q5. What information is required to locate a place on a WebGIS map?

To accurately position and visualize a geographic entity on a WebGIS map canvas, the following technical components are required:

- **1. Numerical Spatial Coordinates (Latitude & Longitude):** The fundamental positional coordinates defining the feature's location (e.g., Faisal Mosque at Latitude `33.7297° N`, Longitude `73.0372° E`).
- **2. Coordinate Reference System (CRS) / Spatial Datum:** The mathematical reference frame modeling the Earth. WebGIS standardizes on **WGS84 (EPSG:4326)** for storage and **Web Mercator (EPSG:3857)** for projectively displaying tiles on 2D screens.
- **3. Map Scale and Zoom Level:** The magnification ratio determining the level of detail (e.g., Zoom 11–13 for city overview, Zoom 16–18 for building detail).
- **4. Cartographic Symbology / Visual Representation:** The graphical entity representing the location, such as custom SVG/DivIcon pins, vectors (polylines/polygons), color styling, and icon badges.
- **5. Attribute Metadata & Semantic Properties:** Descriptive data associated with the feature (e.g., Place Name, Category, Address, Capacity), which Leaflet binds to popups and tooltips.

9. Conclusion & Academic Reflection

This laboratory assignment demonstrated the complete development lifecycle of a modern interactive WebGIS Navigation Dashboard. By integrating **OpenStreetMap** cartography, **Leaflet.js** event-driven mapping, the **HTML5 Geolocation API**, and the **OSRM REST Routing Engine**, the application achieves seamless real-time route calculation and distance/time telemetry for Islamabad's landmark mosques. The use of AI tools significantly accelerated UI styling, architecture design, and API integration.

Submitted by: Emaan Abid (24-ARID-2205) **Course:** RSG-505 WebGIS Development
Department: Institute of Geo-Information and Earth Observation (IGEO), PMAS-AAUR **Instructor:** Sir Jaffar Waqas • Date: October 2026